Assignment 5

Due 2 months ago · Locked 2 months ago · Grades published

Assignment description

In this assignment we will continue work on the interpreter that you started in Assignment 4. This week we will focus on adding variables and statements to our language. More precisely we will

- evaluates arithmetic expressions with integer variables (we have no type system).
- evaluates boolean expressions where the nested arithmetic expressions now can contain variables
- evaluates statements
- mutable state that handles program variables

The code template

- `Language.fs`, which contains a description of the language and error messages (the latter is used for the Yellow assignments)
- `Programs.fs`, which contains a few example programs
- `State.fs`, which contains program state
- `Eval.fs`, which contains the actual interpreter. In this assignment you will continue working on `Eval.fs` that you made in the last assignment.

You need to hand in `State.fs` and `Eval.fs`

Important: To pass this assignment you have to pass either the `Green` or the `Yellow` tests, but not both (they use different types). We recommend that you start with the `Green` ones and then refactor to use the `Yellow` ones.

Doubly important: Remember that all uses of tools like ChatGPT or CoPilot are strictly prohibited and considered academic misconduct. Write your code yourself.

Green Questions

In the file state.fs

Create a function `reservedVariableName` of type `string -> bool` that given a variable name `v` returns true if

- `true` if `v` is one of `if`, `then`, `else`, `while`, `declare`, `print`, `random`, `fork`, or `__result__`.
- `false` otherwise

Hint: use `List.exists`

Examples:

```
> reservedVariableName "hello"
- val it: bool = false

> reservedVariableName "if"
- val it: bool = true

> reservedVariableName "__result__"
- val it: bool = true
```

Create a function `validVariableName` of type `string -> bool` that given a variable name `v` returns true if

- `v` starts with a letter or an underscore (Hint: use `System.Char.IsAsciiLetter`).
- `v` only contains letters, numbers, or underscores (Hint: the functions `String.forall` and `System.Char.IsAsciiLetterOrDigit` are useful)

and `false` otherwise.

Examples:

```
> validVariableName "_hello_1"
- val it: bool = true

> validVariableName "1_hello"
- val it: bool = false
```

Create a type `state`, using either a disjoint union or a record type, that

contains a map from strings to integers that will contain your variables. Do not just use a type alias, like you did for complex numbers, as this will not scale for the rest of the course. Moreover, using a record or disjoint union will make the pretty printing accurate.

Create a function `mkState : unit -> state` that returns a state with an empty variable environment.

Create a function `declare` of type `string -> state -> state option` that given a variable name `x` and a state `st` returns

- `Some st'` where `st'` is equal to `st` with the variable `x` set to `0` if all of the following hold
  - `x` does not appear in the variables of `st`
  - `x` is a valid variable name
  - `x` is not a reserved variable name
- `None` otherwise

In effect this means that you may not declare the same variable twice. We will say that a variable is declared if it exists in the state.

Create a function `getVar` of type `string -> state -> int option` that given a variable name `x` and a state `st` returns

- `Some v`, where `v` is the valuated associated with the variable `x` if that variable is declared in `st`
- `None` otherwise

Create a function `setVar` of type `string -> int -> state -> state option` that given a variable name `x`, an integer value `v` and a state `st` returns

- `Some st'` where `st'` is equal to `st` where the variable `x` has had its value updated to `v`, if `x` is declared in `st`
- `None` otherwise

Examples:

```
open Option

> () |> mkState |> getVar "x"
```

```
-·val·it:·int·option·=·None

>·()·|>·mkState·|>·declare·"x"·|>·bind·(getVar·"x")
-·val·it:·int·option·=·Some·0

>·()·|>·mkState·|>·declare·"x"·|>·bind·(setVar·"x"·42)·|>·bind·
(getVar·"x")
-·val·it:·int·option·=·Some·42

>·()·|>·mkState·|>·declare·"1x"·|>·bind·(setVar·"1x"·42)·|>·bind·
(getVar·"1x")
-·val·it:·int·option·=·None
```

In the file Eval.fs

For all of the functions in this file you may not pattern match on states in any way but only use the functions that you created in `State.fs` to obtain data from and modify state.

Create a recursive function `aexprEval` of type `aexpr -> state -> int option` that given an arithmetic expression `a` and a state `st` works exactly like `aexprEval` from Assignment 4 (so you may not use `Option.bind`) but returns

- `Some x` if a is equal to `Var v` and `v` is declared in `st`
- `None` otherwise

```
open·Option

let·emptyState·=·mkState·()
let·st·=·emptyState·|>·declare·"x"·|>·bind·(setVar·"x"·42)·|>·
Option.get

>·st·|>·aexprEval·(Num·42)
-·val·it:·int·option·=·Some·42

>·st·|>·aexprEval·(Var·"x")
-·val·it:·int·option·=·Some·42
```

```
> st |> aexprEval (Var "y")
- val it: int option = None

> st |> aexprEval (Div (Var "x", Num 2))
- val it: int option = Some 21

> st |> aexprEval (Div (Var "x", Num 0))
- val it: int option = None
```

Create a function `aexprEval2` that behaves exactly the same as `aexprEval` but which uses `Option.bind`. You may not pattern match on options in any way, such as the ones obtained by using recursive calls to `aexprEval2`, and you may not use `aexprEval`.

Create a function bexprEval of the type `bexpr -> state -> bool option` that behaves exactly like `bexprEval` from Assignment 4 but which additionally takes a state as an argument and uses that state in your `aexprEval` or `aexprEval2` functions to compute the value of the arithmetic expressions

Create a function `stmntEval` of type `stmnt -> state -> state option` that given a statement `s` and an initial state `st` returns

- `Some st`, if `s` is equal to `Skip`
- `Some st'`, if `s` is equal to `Declare v`, where `st'` is equal to `st` with the variable `v` declared in `st`,
- `Some st'`, if `s` is equal to `Assign(v, a)`and `a` evaluates to `Some x` in `st`, where `st'` is equal to `st` with the variable `v` set to `x`.
- `Some st''` if `s` is equal to `Seq(s1, s2)`, `s1` evaluates to `Some st'` in state `st`, and `s2` evaluates to `Some st''` in `st'`.
- `Some st'`, if `s` is equal to `If (guard, s1, s2)` and `guard` evaluates to `Some x` in `st`, and if either `x` is equal to
  - `true` and `s1` evaluates to `Some st'` in `st`.
  - `false` and `s2` evaluates to `Some st'` in `st`.
- `Some st''`, if `s` is equal to `While (guard, s')` and `guard` evaluates to `Some x` in `st`, and if either `x` is equal to
  - `true` and `s'` evaluates to `Some st'` in `st` and `While (guard, s')` evaluates to `st''` in `st'.
  - `false` then `st''` is equal to `st`

- `None` otherwise

Yellow Questions

All error handling up to this point has been handled through the `option` type, which either gives you an answer or tells you that something went wrong but you don't know what.

In `Language.fs` you will find the `Error` type that contains possible errors.

Refactor the code above to return a `Result<int, Error>` rather than an `int option` such that they return `Ok result` in stead of `Some result` when the functions succeed and return an `Error`, rather than `None`, such that

- `declare` returns the error
  - `VarAlreadyExists(v)` if the variable `v` has already been declared
  - `ReservedName(v)` if `v` is one of `if`, `then`, `else`, `while`, `declare`, `print`, `random`, `fork`, or `__result__`.
  - `InvalidVarName(v)` if `v` is an invalid variable name.
- `getVar` and `setVar` return `VarNotDeclared v` if the variable `v`, that they are either trying to get or set, has not been declared. You do not have to concern yourself with illegal or invalid names here, `declare` does that.
- `aexprEval` forwards any errors from `setVar` and also, as in Assignment 4, returns `DivisionByZero` if the user attempts to divide or modulo by `0`.
- `bexprEval` forwards any errors from `aexprEval`
- `stmntEval` forwards any errors from `declare`, `setVar`, `aexprEval`, and `bexprEval`.

Hint: The `Result` library also has a `bind` function that is very useful. You will find more useful functions if you read through its documentation.

Red Questions

The current interpreter does not allow variable shadowing. If a variable is declared, it is known forever. In standard programmnig languages variables that are declared inside a while statement, for instance, are not available outside that while statement. In your current implementation, however, they are.

Change your state so that in stead of storing their variables in a map from variables to integers (`Map<string, int>`) you store them in a stack of such maps (`Map<string, int> list`). The idea is that

- when you enter a block from a while- or an if-statement, you push an empty variable environment (an empty map) to the top of the stack and when you exit the block you pop the top element.
- When you read a variable you start reading from the top of the stack, check if the variable is there and if so returns its value, if its not go down one level in the stack and try again

In stack.fs

Change `declare` so that

- it only returns the `VarNotDeclared(v)` error if `v` is declared in the top environment of the stack, meaning that you may re-declare a variable as long as the other identical declarations appear further down the stack.
- puts the newly declared variable at the top of the stack.

Change `getVar` and `setVar` so that they traverse through the stack and get or set the first occurence of the variable they find. Only if they reach the end of the stack do they return the error `VarNotDeclared`.

Create a function `push` of type `state -> state` that given a state `st` pushes an empty variable enviroment (an empty map) to the top of the stack.

Create a function `pop` of type `state -> state` that pops the top element off the variable stack. You do not have to take care of the case where the stack is empty but just let it fail (just like `List.head`) does, for instance.

In Eval.fs

In `stmntEval` change the `If` and `While` cases such that whenever you enter the branches of an if, else, or while statement you push a new empty environment to the stack, and you pop the top one off the stack when you exit the same branch. This effectively means that you will shadow any variable declared outside the branch (as `getVar` and `setVar` will find the newly declared variables first rather than the old ones) and that any variable declared inside a branch will be forgotten once you exit. Every pop should be preceeded

by a push so you should never be able to pop from an empty stack.